EPAM INDIA

QUALITY ENGINEERING INTERVIEW GUIDE

Round 1 + Round 2 • 47 Questions • Layman Explanations • Java / Selenium / API / CI-CD / Agile / SQL

Prepared for a Senior SDET / Test Automation Lead interview

How to use this guide

Each question includes a plain-English explanation, a concrete example, and code where useful. Code blocks use a dark editor-style background with high-contrast monospace text and preserved indentation. Practise speaking the explanation first, then write the code without looking.

Interview date in context: 17 October 2026, 12:00 PM, in person. Prioritise clear reasoning, practical examples, and explaining trade-offs—not memorising code blindly.

Contents

Round 1 — Virtual technical round (1 hour 30 minutes): 20 questions

R1.01 WAP to find the last non-repeating character from a string

R1.02 Use of Comparable and Comparator; write Java code

R1.03 Selenium 4 new features

R1.04 Difference between Page Object and Page Factory

R1.05 Meaning of :: in Java streams

R1.06 Where have you used Java Streams in your framework?

R1.07 How to handle elements inside Shadow DOM

R1.08 How to throw a custom exception in Java

R1.09 Selenium code to handle multiple windows

R1.10 Implement serialization and deserialization

R1.11 Implement OOP concepts for a Savings Bank

R1.12 What is Page Factory Design Pattern?

R1.13 What is Singleton Design Pattern?

R1.14 Where to use LinkedList concepts in automation

R1.15 Exception hierarchy in Java

R1.16 Checked exception examples

R1.17 Fail-fast vs fail-safe iterators

R1.18 When to use ConcurrentHashMap

R1.19 What is ObjectMapper in REST Assured?

R1.20 Meaning of HTTP 405 in Postman and how to resolve it

Round 2 — Technical / process round (1 hour 30 minutes): 27 questions

R2.01 How to execute only regression-related tests in TestNG

R2.02 Read Cucumber table data without Scenario Outline

R2.03 How to perform parallel execution in Cucumber

R2.04 Conditional hooks vs directional hooks in Cucumber

R2.05 CI/CD Jenkins process: how to run Selenium tests

R2.06 Continuous Integration vs Continuous Deployment

R2.07 Custom reporting with Extent Report and Allure

R2.08 Why use logging (Log4j2) in Selenium framework

R2.09 Different Log4j2 log levels

R2.10 Use of Appenders in Log4j2

R2.11 Root causes of Selenium flakiness and fixes

R2.12 Explain Root Cause Analysis techniques

R2.13 Use of git cherry-pick

R2.14 Branching strategy in Git

R2.15 Estimation techniques in Agile

R2.16 Metrics to track project progress in Agile

R2.17 Explain a Test Plan document

R2.18 Difference between Test Plan and Test Strategy

R2.19 Prepare manual test cases and explain test design techniques

R2.20 How to calculate defect leakage

R2.21 Explain defect leakage meaning

R2.22 Explain Test Pyramid

R2.23 DDL vs DML

R2.24 Use of subqueries in SQL

R2.25 Java 8 streams: capitalize car names with length > 3

R2.26 Write IAnnotationTransformer method in TestNG

R2.27 SQL Joins explained

ROUND 1 — VIRTUAL TECHNICAL (90 MINUTES)

R1.01 WAP to find the last non-repeating character from a string

Layman explanation: Count how many times each character occurs, then scan the string from right to left. The first character with count 1 is the last non-repeating character.

Example: Example: "swiss" → last non-repeating character is "w". If none exists, return an empty result or explain that no such character exists.

CODE / ILLUSTRATION

```
import java.util.*;

public class LastUniqueChar {
    public static Character lastNonRepeating(String s) {
        Map<Character, Integer> counts = new HashMap<>();

        for (char c : s.toCharArray()) {
            counts.put(c, counts.getOrDefault(c, 0) + 1);
        }

        for (int i = s.length() - 1; i >= 0; i--) {
            char c = s.charAt(i);
            if (counts.get(c) == 1) {
                return c;
            }
        }
        return null;
    }
}
```

Interview tip: Interview note: clarify whether spaces and letter case count as characters. This solution is O(n) time and O(k) space.

R1.02 Use of Comparable and Comparator; write Java code

Layman explanation: Comparable defines a class's natural/default order using compareTo(). Comparator defines an external/custom order and lets you have multiple sorting rules for the same class.

Example: Example: Employee natural order by ID; custom order by salary descending.

CODE / ILLUSTRATION

```
import java.util.*;

class Employee implements Comparable<Employee> {
    int id;
    String name;
    double salary;

    Employee(int id, String name, double salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }

    public int compareTo(Employee other) {
        return Integer.compare(this.id, other.id); // natural order
    }
}

class Demo {
    public static void main(String[] args) {
        List<Employee> list = new ArrayList<>();
        list.add(new Employee(2, "Maya", 80000));
        list.add(new Employee(1, "Amit", 90000));

        Collections.sort(list); // by ID
        list.sort(Comparator.comparingDouble(
            (Employee e) -> e.salary).reversed()); // salary high to low
    }
}
```

Interview tip: Avoid subtraction such as id - other.id because integer overflow is possible. Use Integer.compare().

R1.03 Selenium 4 new features

Layman explanation: Selenium 4 aligns with the W3C WebDriver standard and adds relative locators, a redesigned Selenium Grid, new window/tab APIs, DevTools integration, and improved element interactions.

Example: Examples: open a new tab with driver.switchTo().newWindow(WindowType.TAB); locate an element above/near another using RelativeLocator; use Grid 4 for distributed execution.

CODE / ILLUSTRATION

```
// Open a new tab
driver.switchTo().newWindow(WindowType.TAB);
driver.get("https://example.com");

// Relative locator: element to the right of a label
WebElement label = driver.findElement(By.id("emailLabel"));
WebElement field = driver.findElement(
    RelativeLocator.with(By.tagName("input")).toRightOf(label));
```

Interview tip: Mention the feature relevant to the problem, not just a memorised list.

R1.04 Difference between Page Object and Page Factory

Layman explanation: Page Object Model (POM) is a design approach: each page/component has a class containing locators and actions. PageFactory is a Selenium helper that initializes fields annotated with @FindBy, commonly used inside a Page Object.

Example: POM is the design pattern; PageFactory is one optional way to initialise page elements. POM does not require PageFactory.

CODE / ILLUSTRATION

```
// Plain Page Object
public class LoginPage {
    private WebDriver driver;
    private By username = By.id("username");

    public LoginPage(WebDriver driver) {
        this.driver = driver;
    }

    public void enterUsername(String value) {
        driver.findElement(username).sendKeys(value);
    }
}

// PageFactory style
public class LoginPageFactory {
    @FindBy(id = "username")
    private WebElement username;
}
```

Interview tip: PageFactory does not replace POM. Modern frameworks often prefer explicit By locators for clearer timing and element lookup.

R1.05 Meaning of :: in Java streams

Layman explanation: The :: operator is a method reference. It points to an existing method and is a shorter alternative to a lambda when the method signature matches.

Example: Example: names.forEach(System.out::println) is equivalent to names.forEach(name -> System.out.println(name)).

CODE / ILLUSTRATION

```
List<String> names = Arrays.asList("Asha", "Ravi");
names.forEach(System.out::println);

List<String> upper = names.stream()
    .map(String::toUpperCase)
    .collect(Collectors.toList());
```

Interview tip: Common forms: ClassName::staticMethod, object::instanceMethod, ClassName::instanceMethod, and ClassName::new.

R1.06 Where have you used Java Streams in your framework?

Layman explanation: Use Streams for concise collection processing—for example, filtering test data, extracting response values, transforming lists, or checking whether expected values exist.

Example: Example: from API response objects, extract all user IDs; or filter test cases by tag/status. Do not use Streams just to make simple code look clever.

CODE / ILLUSTRATION

```
List<String> activeNames = users.stream()
    .filter(User::isActive)
    .map(User::getName)
    .collect(Collectors.toList());

boolean hasExpectedStatus = responseStatuses.stream()
    .anyMatch("ACTIVE"::equals);
```

Interview tip: A strong answer names a real framework use case and explains why it improved readability or reduced boilerplate.

R1.07 How to handle elements inside Shadow DOM

Layman explanation: A Shadow DOM element lives inside a component's shadow root. In Selenium 4, locate the host element, get its shadow root, and find the inner element from that root.

Example: Example: a custom web component contains an input that ordinary page-level locators cannot see.

CODE / ILLUSTRATION

```
WebElement host = driver.findElement(By.cssSelector("my-login"));
SearchContext shadowRoot = host.getShadowRoot();
WebElement input = shadowRoot.findElement(By.cssSelector("#username"));
input.sendKeys("swapnil");
```

Interview tip: For nested shadow roots, repeat host → getShadowRoot() → find inner host. Closed shadow roots may not be accessible through standard WebDriver APIs.

R1.08 How to throw a custom exception in Java

Layman explanation: Create a class that extends Exception for a checked exception or RuntimeException for an unchecked exception, then throw an instance when a meaningful business/framework condition occurs.

Example: Example: throw TestDataNotFoundException when required test data is missing.

CODE / ILLUSTRATION

```
class TestDataNotFoundException extends RuntimeException {
    public TestDataNotFoundException(String message) {
        super(message);
    }
}

if (testData == null) {
    throw new TestDataNotFoundException(
        "Required test data was not found");
}
```

Interview tip: Do not catch and ignore exceptions. Include context that helps diagnose the failure.

R1.09 Selenium code to handle multiple windows

Layman explanation: Store the original window handle, trigger the new window, switch to the handle that differs from the original, perform actions, then switch back.

Example: Example: click a link that opens a payment provider in another tab.

CODE / ILLUSTRATION

```
String parent = driver.getWindowHandle();

driver.findElement(By.id("openPayment")).click();

for (String handle : driver.getWindowHandles()) {
    if (!handle.equals(parent)) {
        driver.switchTo().window(handle);
        break;
    }
}

System.out.println(driver.getTitle());
driver.close();                 // closes current child window
driver.switchTo().window(parent);
```

Interview tip: In robust tests, wait until the number of windows increases before switching, and do not assume the child is always the second handle.

R1.10 Implement serialization and deserialization

Layman explanation: Serialization converts an object into a byte stream so it can be stored or transmitted. Deserialization reconstructs the object from that stream. Java native serialization uses Serializable.

Example: Example: save a simple User object to a file and read it back. In real applications, use trusted input only; native deserialization of untrusted data is dangerous.

CODE / ILLUSTRATION

```
import java.io.*;

class User implements Serializable {
    private static final long serialVersionUID = 1L;
    String name;
    User(String name) { this.name = name; }
}

class Demo {
    public static void main(String[] args) throws Exception {
        try (ObjectOutputStream out = new ObjectOutputStream(
                new FileOutputStream("user.ser"))) {
            out.writeObject(new User("Asha"));
        }

        try (ObjectInputStream in = new ObjectInputStream(
                new FileInputStream("user.ser"))) {
            User user = (User) in.readObject();
            System.out.println(user.name);
        }
    }
}
```

Interview tip: transient fields are skipped by default. serialVersionUID helps with version compatibility.

R1.11 Implement OOP concepts for a Savings Bank

Layman explanation: Model an abstract BankAccount, hide the balance using encapsulation, inherit SavingsAccount from it, and use polymorphism for account-specific behaviour. Abstraction exposes essential operations while hiding implementation details.

Example: Example: deposit and withdraw are common operations; a savings account may enforce a minimum balance.

CODE / ILLUSTRATION

```
abstract class BankAccount {
    private double balance;

    BankAccount(double openingBalance) {
        if (openingBalance < 0) throw new IllegalArgumentException();
        balance = openingBalance;
    }

    public void deposit(double amount) {
        if (amount <= 0) throw new IllegalArgumentException();
        balance += amount;
    }

    public double getBalance() { return balance; }

    public abstract boolean withdraw(double amount);
}

class SavingsAccount extends BankAccount {
    private static final double MIN_BALANCE = 1000;

    SavingsAccount(double openingBalance) { super(openingBalance); }

    @Override
    public boolean withdraw(double amount) {
        if (amount <= 0 || getBalance() - amount < MIN_BALANCE) {
            return false;
        }
        // In a production model, balance mutation should be implemented
        // through a protected domain operation in the base class.
        return true;
    }
}
```

Interview tip: For a fully working implementation, provide a protected debit operation in the base class and call it after validation. Explain the four OOP pillars clearly.

R1.12 What is Page Factory Design Pattern?

Layman explanation: PageFactory is a Selenium support utility—not a separate core GoF design pattern—that initializes WebElement fields annotated with @FindBy, typically through PageFactory.initElements().

Example: Example: @FindBy(id = "login") private WebElement loginButton; reduces repeated findElement calls in page classes.

CODE / ILLUSTRATION

```
public class LoginPage {
    @FindBy(id = "login")
    private WebElement loginButton;

    public LoginPage(WebDriver driver) {
        PageFactory.initElements(driver, this);
    }

    public void login() {
        loginButton.click();
    }
}
```

Interview tip: Mention that POM is the architecture/design approach; PageFactory is an optional element-initialization mechanism.

R1.13 What is Singleton Design Pattern?

Layman explanation: Singleton ensures that a class has one shared instance and provides a single access point to it. It is useful for genuinely shared, stateless resources, but a global WebDriver singleton can cause test interference in parallel execution.

Example: Example: one immutable configuration object. For parallel Selenium tests, prefer ThreadLocal WebDriver or dependency injection rather than one global driver.

CODE / ILLUSTRATION

```
public final class Config {
    private static final Config INSTANCE = new Config();

    private Config() { }

    public static Config getInstance() {
        return INSTANCE;
    }
}
```

Interview tip: A private constructor prevents external construction; a static accessor returns the shared instance.

R1.14 Where to use LinkedList concepts in automation

Layman explanation: LinkedList is useful when frequent insertions/removals at the ends are needed, such as a queue of pending work or a deque for processing tasks. It is not automatically better than ArrayList.

Example: Example: use Deque for queued retry jobs or breadth-first traversal of dependent test tasks. For random indexed access, ArrayList is usually better.

CODE / ILLUSTRATION

```
Deque<String> pendingJobs = new LinkedList<>();
pendingJobs.addLast("smoke");
pendingJobs.addLast("regression");

String nextJob = pendingJobs.removeFirst();
```

Interview tip: Choose the collection based on the operation pattern; avoid claiming LinkedList is commonly needed for ordinary test data.

R1.15 Exception hierarchy in Java

Layman explanation: Throwable is the root. It has Error and Exception branches. RuntimeException is a subclass of Exception; checked exceptions are generally Exception types other than RuntimeException and its subclasses.

Example: Examples: IOException is checked; NullPointerException is unchecked; OutOfMemoryError is an Error and usually should not be handled as a normal application exception.

CODE / ILLUSTRATION

```
Throwable
 ■■■ Error
 ■■■ Exception
      ■■■ RuntimeException (unchecked)
      ■    ■■■ NullPointerException
      ■    ■■■ IllegalArgumentException
      ■■■ Other checked exceptions
           ■■■ IOException
           ■■■ SQLException
```

Interview tip: Errors generally indicate serious JVM/system problems; exceptions represent conditions an application may be able to handle.

R1.16 Checked exception examples

Layman explanation: Checked exceptions are checked by the compiler; code must catch them or declare them with throws. Common examples are IOException, SQLException, and ClassNotFoundException.

Example: Example: reading a file may fail because it does not exist or access is denied.

CODE / ILLUSTRATION

```
public String readFile(String path) throws IOException {
    return Files.readString(Path.of(path));
}

// Caller must catch IOException or declare it too.
```

Interview tip: InterruptedException is also checked. Do not confuse it with unchecked NullPointerException.

R1.17 Fail-fast vs fail-safe iterators

Layman explanation: Fail-fast iterators usually throw ConcurrentModificationException when a collection is structurally modified during iteration outside the iterator's supported methods. 'Fail-safe' is informal terminology often used for snapshot or weakly consistent iterators.

Example: Example: ArrayList iterator is fail-fast; CopyOnWriteArrayList iterates over a snapshot and allows concurrent modifications.

CODE / ILLUSTRATION

```
List<String> list = new ArrayList<>(Arrays.asList("A", "B"));
for (String item : list) {
    // list.add("C"); // can throw ConcurrentModificationException
}

CopyOnWriteArrayList<String> safe =
    new CopyOnWriteArrayList<>(Arrays.asList("A", "B"));
```

Interview tip: Fail-fast is best-effort, not a thread-safety guarantee. CopyOnWriteArrayList can be expensive when writes are frequent.

R1.18 When to use ConcurrentHashMap

Layman explanation: Use ConcurrentHashMap when multiple threads need to read and update a shared map concurrently without synchronizing every operation on the entire map.

Example: Example: shared counters/cache in a parallel test service. Avoid sharing mutable test-specific state unnecessarily.

CODE / ILLUSTRATION

```
ConcurrentMap<String, Integer> counts = new ConcurrentHashMap<>();
counts.merge("passed", 1, Integer::sum);

Integer value = counts.get("passed");
```

Interview tip: Individual operations are thread-safe; a sequence of multiple operations may still need atomic methods such as computeIfAbsent or merge.

R1.19 What is ObjectMapper in REST Assured?

Layman explanation: ObjectMapper is usually Jackson's class for converting Java objects to JSON and JSON back to Java objects. REST Assured can use Jackson for POJO serialization/deserialization when configured with the dependency.

Example: Example: serialize a request POJO to JSON or map a JSON response to a Java class.

CODE / ILLUSTRATION

```
ObjectMapper mapper = new ObjectMapper();

String json = mapper.writeValueAsString(requestPojo);
User user = mapper.readValue(json, User.class);

// REST Assured can also serialize a POJO:
given()
    .contentType(ContentType.JSON)
    .body(requestPojo)
.when()
    .post("/users");
```

Interview tip: ObjectMapper belongs to Jackson, not REST Assured itself. Ensure the Jackson dependency and content type are correct.

R1.20 Meaning of HTTP 405 in Postman and how to resolve it

Layman explanation: 405 Method Not Allowed means the server recognises the HTTP method but that method is not allowed for the target resource. Check the endpoint's supported methods and API contract.

Example: Example: sending POST to a read-only endpoint that supports GET. Use the correct method, URL, route, and headers; check the Allow response header if provided.

CODE / ILLUSTRATION

```
GET    /users/10       // retrieve
PUT    /users/10       // replace/update, if supported
DELETE /users/10       // delete, if supported
```

Interview tip: Do not blindly change the method; confirm the API specification and server routing.

ROUND 2 — TECHNICAL / PROCESS (90 MINUTES)

R2.01 How to execute only regression-related tests in TestNG

Layman explanation: Tag regression tests using groups, then include only that group in the TestNG XML or Maven configuration.

Example: Example: tests marked @Test(groups = "regression") run when the regression group is included.

CODE / ILLUSTRATION

```
@Test(groups = {"regression"})
public void updateUserTest() { }

// testng.xml
<suite name="RegressionSuite">
  <test name="Regression">
    <groups>
      <run><include name="regression"/></run>
    </groups>
    <classes>
      <class name="tests.UserTests"/>
    </classes>
  </test>
</suite>
```

Interview tip: Keep group names consistent and verify that smoke-only tests are not accidentally included.

R2.02 Read Cucumber table data without Scenario Outline

Layman explanation: Use a DataTable parameter in a step definition when a normal scenario contains a table. Scenario Outline is for repeating a scenario with multiple Examples rows.

Example: Example: a Given step receives a table of usernames and roles.

CODE / ILLUSTRATION

```
Given("the following users exist:", (DataTable table) -> {
    List<Map<String, String>> users = table.asMaps();
    for (Map<String, String> user : users) {
        System.out.println(user.get("username"));
    }
});
```

Interview tip: Use asMaps() for a header row; use asLists() when the table has no header.

R2.03 How to perform parallel execution in Cucumber

Layman explanation: Use a supported parallel runner/configuration and ensure scenarios do not share mutable state. With TestNG, a Cucumber TestNG runner can enable parallel scenarios through DataProvider(parallel = true).

Example: Example: independent scenarios run in separate threads; each scenario needs its own driver and scenario context.

CODE / ILLUSTRATION

```
@CucumberOptions(
    features = "src/test/resources/features",
    glue = "steps"
)
public class TestRunner extends AbstractTestNGCucumberTests {
    @Override
    @DataProvider(parallel = true)
    public Object[][] scenarios() {
        return super.scenarios();
    }
}
```

Interview tip: Set thread count in suite/configuration, use ThreadLocal or scenario-scoped driver management, and avoid shared test data collisions.

R2.04 Conditional hooks vs directional hooks in Cucumber

Layman explanation: Conditional hooks are hooks with a tag condition, such as @Before("@smoke"). 'Directional hooks' is not a standard official Cucumber term; interviewers may mean hook order or before/after direction. Clarify what they mean.

Example: Example: run setup only for @api scenarios; use @Before(order = 1) to control order and @After for cleanup.

CODE / ILLUSTRATION

```
@Before("@api")
public void setupApi() {
    // API-specific setup
}

@Before(order = 1)
public void commonSetup() {
    // lower order runs earlier
}

@After
public void cleanup() {
    // cleanup after scenario
}
```

Interview tip: Cucumber hook ordering rules can vary by hook type/version; check the version used by the framework.

R2.05 CI/CD Jenkins process: how to run Selenium tests

Layman explanation: A Jenkins pipeline checks out code, installs/uses the required JDK and browser/driver environment, builds with Maven, runs tagged tests, publishes reports, and archives logs/screenshots.

Example: Example: run mvn clean test -Dgroups=regression; publish Surefire and Allure results even when tests fail.

CODE / ILLUSTRATION

```
pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps { checkout scm }
    }
    stage('Run UI tests') {
      steps { sh 'mvn clean test -Dgroups=regression' }
    }
  }
  post {
    always {
      junit '**/target/surefire-reports/*.xml'
      archiveArtifacts artifacts: '**/screenshots/**', allowEmptyArchive: true
    }
  }
}
```

Interview tip: For Windows agents use bat instead of sh. In production, configure tool versions, secrets, browser environment, and Allure publishing.

R2.06 Continuous Integration vs Continuous Deployment

Layman explanation: Continuous Integration (CI) frequently merges code and runs automated build/tests. Continuous Delivery keeps changes ready for release, often with a manual production approval. Continuous Deployment automatically releases every qualifying change to production.

Example: Example: CI runs tests on each pull request; deployment may require an approved release gate or may be fully automatic.

Interview tip: Remember the distinction between continuous delivery and continuous deployment; they are related but not identical.

R2.07 Custom reporting with Extent Report and Allure

Layman explanation: Both provide readable test results. Extent lets you create a customised report and log steps directly. Allure collects test results and attachments, then generates a report using its adapter/CLI.

Example: Example: log a passed step, attach a screenshot on failure, and publish the report from CI.

CODE / ILLUSTRATION

```
// Extent Reports (illustrative)
ExtentReports extent = new ExtentReports();
ExtentTest test = extent.createTest("Login test");
test.info("Opening login page");
test.pass("Login succeeded");
extent.flush();

// Allure attachment
@Attachment(value = "Screenshot", type = "image/png")
public byte[] attachScreenshot(byte[] screenshot) {
    return screenshot;
}

// Typical Maven command when Allure results are configured:
// mvn clean test
// allure serve target/allure-results
```

Interview tip: Exact setup depends on TestNG/JUnit/Cucumber adapters and framework version. Always preserve failure details and screenshots/logs.

R2.08 Why use logging (Log4j2) in Selenium framework

Layman explanation: Logging records what the test was doing, when it happened, and why it failed. It is more useful than scattered System.out.println calls, especially in CI and parallel execution.

Example: Example: log test start, URL, key business actions, API response status, retry reason, and exception details; avoid passwords/tokens.

CODE / ILLUSTRATION

```
private static final Logger log =
    LogManager.getLogger(LoginTest.class);

log.info("Starting login test");
log.debug("Navigating to login page");
log.warn("Element took longer than expected");
log.error("Login failed", exception);
```

Interview tip: Use structured, useful messages and mask secrets or personal data.

R2.09 Different Log4j2 log levels

Layman explanation: Common levels from most detailed to most severe are TRACE, DEBUG, INFO, WARN, ERROR, and FATAL; OFF disables logging and ALL enables all levels.

Example: Example: DEBUG for locator diagnostics, INFO for test milestones, WARN for recoverable issues, ERROR for failed operations.

Interview tip: The configured threshold controls which messages are emitted; e.g. INFO typically excludes DEBUG and TRACE.

R2.10 Use of Appenders in Log4j2

Layman explanation: An Appender decides where log events go, such as the console, a file, a rolling file, or a remote destination. Layouts control formatting.

Example: Example: Console appender for local debugging and RollingFile appender to rotate logs in CI.

CODE / ILLUSTRATION

```
<Configuration status="WARN">
  <Appenders>
    <Console name="Console" target="SYSTEM_OUT">
      <PatternLayout pattern="%d %-5p [%t] %c - %m%n"/>
    </Console>
    <RollingFile name="File"
        fileName="logs/test.log"
        filePattern="logs/test-%d{yyyy-MM-dd}-%i.log.gz">
      <PatternLayout pattern="%d %-5p [%t] %c - %m%n"/>
      <Policies>
        <SizeBasedTriggeringPolicy size="10 MB"/>
      </Policies>
    </RollingFile>
  </Appenders>
  <Loggers>
    <Root level="info">
      <AppenderRef ref="Console"/>
      <AppenderRef ref="File"/>
    </Root>
  </Loggers>
</Configuration>
```

Interview tip: Appender = destination; Layout = message format; Logger = emits events; Filter = decides which events pass.

R2.11 Root causes of Selenium flakiness and fixes

Layman explanation: Flaky tests pass and fail without a relevant product change. Common causes include timing/race conditions, unstable locators, shared state, test-data collisions, environment/network instability, animations, and poor cleanup.

Example: Fix with explicit condition-based waits, stable test IDs, isolated data, independent tests, deterministic setup/teardown, controlled environments, and better diagnostics.

CODE / ILLUSTRATION

```
WebDriverWait wait = new WebDriverWait(
    driver, Duration.ofSeconds(10));

WebElement button = wait.until(
    ExpectedConditions.elementToBeClickable(
        By.cssSelector("[data-testid='submit']")));
button.click();
```

Interview tip: Avoid Thread.sleep as a default fix and avoid blind retries that hide product defects. Track flakiness rate and investigate patterns.

R2.12 Explain Root Cause Analysis techniques

Layman explanation: RCA finds the underlying cause rather than treating only the symptom. Common techniques are 5 Whys, fishbone/Ishikawa diagram, Pareto analysis, timeline/log analysis, and fault-tree analysis.

Example: Example: test timed out → element was not ready → page request was slow → shared test environment was overloaded; the corrective action should address the actual bottleneck.

Interview tip: State evidence, root cause, corrective action, preventive action, and how you will verify recurrence is reduced.

R2.13 Use of git cherry-pick

Layman explanation: git cherry-pick applies the change introduced by a specific commit onto the current branch without merging the entire source branch.

Example: Example: move a hotfix commit from develop to release branch. Resolve conflicts, test, and avoid cherry-picking the same change repeatedly.

CODE / ILLUSTRATION

```
git switch release/1.2
git cherry-pick a1b2c3d

# If conflicts occur:
git status
# resolve files, then:
git add <resolved-files>
git cherry-pick --continue

# Cancel the operation:
git cherry-pick --abort
```

Interview tip: It creates a new commit on the destination branch. Use it selectively and communicate duplicated commit history where relevant.

R2.14 Branching strategy in Git

Layman explanation: A branching strategy defines how teams isolate work and merge it safely. Common approaches include trunk-based development, GitHub Flow, and Git Flow.

Example: Example: feature branch → pull request/code review → CI checks → merge to main; release branches may be used when the product needs controlled releases.

Interview tip: For frequent delivery, trunk-based development with short-lived branches is often simpler. Choose based on release governance and team needs.

R2.15 Estimation techniques in Agile

Layman explanation: Common techniques include Planning Poker/story points, T-shirt sizing, ideal hours, affinity estimation, and capacity/velocity-based forecasting.

Example: Example: compare a new API test story to previously completed stories and assign 3 points if its complexity is similar.

Interview tip: Story points estimate relative effort/complexity/uncertainty, not exact hours. Include dependencies, test data, automation, review, and defect retesting.

R2.16 Metrics to track project progress in Agile

Layman explanation: Useful metrics include sprint burndown/burnup, velocity trend, cycle time, lead time, throughput, escaped defects, test pass rate, automation stability, and blocked work.

Example: Example: a rising escaped-defect rate plus a falling pass rate may signal quality risk; velocity alone does not prove product quality.

Interview tip: Use metrics to improve the system, not to rank individuals. Explain trend, context, and action.

R2.17 Explain a Test Plan document

Layman explanation: A test plan describes the scope, objectives, approach, resources, schedule, environment, test data, entry/exit criteria, deliverables, risks, and responsibilities for a testing effort.

Example: Example: for a release, define which features and browsers are in scope, who tests them, what blocks release, and how results are reported.

Interview tip: A plan can be tailored to the project; it should be a living document, not paperwork that no one uses.

R2.18 Difference between Test Plan and Test Strategy

Layman explanation: A test strategy describes the higher-level testing approach and principles, often shared across a product or organisation. A test plan applies that approach to a particular project, release, or test cycle.

Example: Example: strategy says use API tests early and risk-based regression; the release plan lists exact features, dates, environments, owners, and exit criteria.

Interview tip: Organisations sometimes combine these documents; explain the purpose rather than insisting on one universal template.

R2.19 Prepare manual test cases and explain test design techniques

Layman explanation: Write test cases with ID, preconditions, steps, test data, expected result, and priority. Techniques include equivalence partitioning, boundary-value analysis, decision tables, state transitions, and pairwise testing.

Example: Example: age field accepts 18–60. Test below boundary 17, boundaries 18 and 60, and above boundary 61; also test valid/invalid formats.

CODE / ILLUSTRATION

```
Test Case: Login with valid credentials
Precondition: User account exists and is active
Steps:
1. Open login page
2. Enter valid username
3. Enter valid password
4. Click Login
Expected result:
User reaches the dashboard and sees their account name

Boundary example for age 18–60:
17 = invalid, 18 = valid, 60 = valid, 61 = invalid
```

Interview tip: Cover positive, negative, boundary, validation, security-relevant, accessibility, and error-handling paths based on risk.

R2.20 How to calculate defect leakage

Layman explanation: Defect leakage is the number of defects discovered after a testing phase or release that should reasonably have been found earlier. One common rate is escaped defects divided by total defects found across the defined phases, multiplied by 100.

Example: Example: 8 escaped defects and 92 defects found before release gives 8 / (92 + 8) × 100 = 8%. Define the denominator consistently.

CODE / ILLUSTRATION

```
Defect leakage rate (%) =
    Escaped defects
    ------------------------------ x 100
    Pre-release defects + escaped defects
```

Interview tip: Teams use different definitions; state whether leakage means post-release only or defects escaping one test phase to the next.

R2.21 Explain defect leakage meaning

Layman explanation: Defect leakage means a bug escaped the phase where it was expected to be detected—for example, it was not found in QA but was discovered in production.

Example: Example: a checkout tax calculation bug is discovered by a customer after release. Investigate missing scenarios, data, environment differences, or review gaps.

Interview tip: It is a signal for process improvement, not automatically proof that an individual tester failed.

R2.22 Explain Test Pyramid

Layman explanation: The test pyramid recommends many fast unit tests at the base, fewer integration/service tests in the middle, and a smaller number of slower end-to-end UI tests at the top.

Example: Example: validate tax rules with unit tests, API integration with service tests, and a few critical checkout journeys through the browser.

Interview tip: Balance matters: a pyramid is a heuristic, not a rigid ratio. Keep UI tests focused on critical user journeys.

R2.23 DDL vs DML

Layman explanation: DDL (Data Definition Language) changes database structure; DML (Data Manipulation Language) reads or changes table data. SELECT is often classified separately as DQL, though some interview contexts group it with DML.

Example: Examples: CREATE/ALTER/DROP are DDL; INSERT/UPDATE/DELETE are DML.

CODE / ILLUSTRATION

```
CREATE TABLE Employee (
    id INT PRIMARY KEY,
    name VARCHAR(100)
); -- DDL

INSERT INTO Employee (id, name)
VALUES (1, 'Asha'); -- DML

UPDATE Employee SET name = 'Ravi' WHERE id = 1; -- DML
```

Interview tip: Be careful with DELETE (rows), DROP (object), and TRUNCATE (remove all rows with database-specific semantics).

R2.24 Use of subqueries in SQL

Layman explanation: A subquery is a query nested inside another query. It can provide a value, a list, or a set of rows for the outer query.

Example: Example: find employees whose salary is above the company average.

CODE / ILLUSTRATION

```
SELECT employee_name, salary
FROM Employee
WHERE salary > (
    SELECT AVG(salary)
    FROM Employee
);
```

Interview tip: Subqueries can appear in WHERE, FROM, or SELECT; use EXISTS for existence checks and compare performance with joins when appropriate.

R2.25 Java 8 streams: capitalize car names with length > 3

Layman explanation: Stream the car list, filter names longer than three characters, transform those names to uppercase, and collect the result. Clarify whether shorter names should be removed or left unchanged; this example filters them out.

Example: Example: ["BMW", "Honda", "Audi", "Kia"] → ["HONDA", "AUDI"].

CODE / ILLUSTRATION

```
List<String> cars = Arrays.asList("BMW", "Honda", "Audi", "Kia");

List<String> result = cars.stream()
    .filter(car -> car.length() > 3)
    .map(String::toUpperCase)
    .collect(Collectors.toList());

System.out.println(result); // [HONDA, AUDI]
```

Interview tip: If the interviewer means preserve every car but capitalise only names longer than 3, use map with a conditional instead of filter.

R2.26 Write IAnnotationTransformer method in TestNG

Layman explanation: IAnnotationTransformer lets you modify TestNG annotations at runtime—for example, to set retry analyzers or alter enabled/priority settings. Its transform method receives the annotation and test metadata.

Example: Example: attach a RetryAnalyzer to every @Test method, then register the transformer in testng.xml or using the supported TestNG configuration.

CODE / ILLUSTRATION

```
public class RetryTransformer implements IAnnotationTransformer {
    @Override
    public void transform(ITestAnnotation annotation,
                          Class testClass,
                          Constructor testConstructor,
                          Method testMethod) {
        annotation.setRetryAnalyzer(RetryAnalyzer.class);
    }
}
```

Interview tip: Use the exact method signature supported by your TestNG version. Register the transformer correctly; retry only transient failures and report retries.

R2.27 SQL Joins explained

Layman explanation: A JOIN combines rows from tables using a related column. INNER JOIN returns matching rows; LEFT JOIN keeps every left-table row and fills unmatched right-side values with NULL; RIGHT JOIN does the reverse; FULL OUTER JOIN keeps unmatched rows from both sides where supported.

Example: Example: Employee.department_id joins Department.id to display employee names and department names.

CODE / ILLUSTRATION

```
SELECT e.name, d.department_name
FROM Employee e
INNER JOIN Department d
    ON e.department_id = d.id;

-- Keep employees even when they have no matching department
SELECT e.name, d.department_name
FROM Employee e
LEFT JOIN Department d
    ON e.department_id = d.id;
```

Interview tip: CROSS JOIN returns combinations of all rows. Self JOIN joins a table to itself. Join keys and duplicate rows matter.

Final revision checklist

■ Explain your approach before coding; state assumptions and edge cases.

■ For Java coding, cover empty input, null where applicable, duplicates, and expected complexity.

■ For Selenium, explain waits, stable locators, driver lifecycle, and parallel safety.

■ For framework questions, connect the answer to a real project example and explain the trade-off.

■ For Agile/process questions, give a definition, a concrete example, a metric or artefact, and the action taken.

■ Never hide failures with broad catch blocks or blind retries; preserve logs, screenshots, and root-cause evidence.

■ When an interviewer says something ambiguous (for example, 'directional hooks'), clarify the intended meaning politely.

Important version note

Some APIs and configuration signatures differ by Selenium, Cucumber, TestNG, Jackson, and Log4j2 version. The code is intended as interview-ready illustration; confirm exact imports/dependencies and signatures against the framework version used in the interview.